

Terasic DE1-SoC Setup Manual

Booting Linux on the Cyclone V HPS — from SD card to a shell

VELEUČILIŠTE U KARLOVCU

Karlovac University of Applied Sciences

About this manual

A practical guide to bringing up Linux on the Terasic DE1-SoC hard processor system and using it for embedded and FPGA development — flashing the card, the serial console, first boot, cross-compilation, and building and reconfiguring your own FPGA designs.

Author	Vedran Vyroubal
Contact	vedran.vyroubal [at] vuka.hr
Institution	Karlovac University of Applied Sciences Veleučilište u Karlovcu
Address	Trg J. J. Strossmayera 9, 47000 Karlovac, Hrvatska
Release	v2.0.2 (2026-09-30) — release page
System	Debian 13.7 (trixie), Linux 6.12

Created at Karlovac University of Applied Sciences (Veleučilište u Karlovcu).

Contents

Overview	3
1 What you need	3
2 Get the image	3
2.1 Option A — download a prebuilt release (quickest)	3
2.2 Option B — build from source (this repo)	4
3 Flash the microSD card	4
3.1 Option A — balenaEtcher (any OS)	4
3.2 Option B — dd (Linux / macOS)	4
4 Set the boot-mode switch	5
5 Connect and power on	5
6 Open the serial console	5
7 First boot and login	6
8 Networking and SSH	6
9 The U-Boot prompt	6
10 Cross-compile for the board	7
11 Build your own FPGA design	7
11.1 Getting the tools	7
12 Changing the FPGA fabric	8
13 Troubleshooting	9

Overview

A from-scratch guide to running modern Linux on the DE1-SoC's hard processor system (HPS) and reaching a shell over serial and SSH — the foundation for embedded and FPGA development on the board. You will end up with **Debian 13.7 (trixie)** on **mainline Linux 6.12**, a serial console at 115200 8N1, and SSH access, with the FPGA fabric configured at boot. The image is headless (no graphical desktop); you work over the serial console and SSH.

Which release this manual describes

This edition describes project release **v2.0.2** (2026-09-30): Debian 13.7 (trixie) on Linux 6.12. Newer releases may differ; the manual shipped with each release matches it (see the release page on GitHub).

1 What you need

Most of this ships in the box; the only items you may need to source are a microSD card and a card reader.

- **DE1-SoC board + PSU** — the 12 V barrel-jack adapter included with the board.
- **microSD card, ≥ 2 GB** — the image ships ~ 1.8 GB and grows to fill the card. All data on it will be erased.
- **microSD card reader** — USB reader or a built-in slot.
- **USB cable (console)** — for the board's UART-to-USB port (Silicon Labs CP210x bridge).
- **Ethernet cable** (optional) — to a network with DHCP, for SSH and internet.
- **Linux build host** (optional) — only if you want to build the image from source rather than flash a prebuilt one: a Debian/Ubuntu x86-64 machine, or any Linux x86-64 machine with Docker.

Software on your computer: an SD-card writer ([balenaEtcher](#), or `dd` on Linux/macOS); a serial terminal (minicom, picocom, screen; PuTTY or Tera Term on Windows); and the Silicon Labs CP210x VCP driver if your OS does not detect the console port automatically.

2 Get the image

This repository builds the image from source — Debian 13 + Linux 6.12 with a vendor-hybrid boot chain (the proven vendor preloader/U-Boot plus a mainline kernel and Debian userspace). **You do not need Quartus:** the compiled FPGA bitstream (`soc_system.rbf`) is committed.

2.1 Option A — download a prebuilt release (quickest)

Download `de1soc-debian13-6.12.img.xz` and `SHA256SUMS.txt` from the [latest release](#) on GitHub, check it and decompress it:

```
sha256sum -c SHA256SUMS.txt --ignore-missing # -> de1soc-debian13-6.12.img.xz: OK
unxz de1soc-debian13-6.12.img.xz             # -> de1soc-debian13-6.12.img
```

Then flash `de1soc-debian13-6.12.img` (next section). ([balenaEtcher](#) can also flash the `.img.xz` directly.)

2.2 Option B — build from source (this repo)

On a Debian/Ubuntu x86-64 host with an internet connection:

```
git clone --recurse-submodules https://github.com/vvyroubal/de1-soc.git DE1-SoC
cd DE1-SoC/hps/newimage
make deps      # one-time: install the host build tools (uses sudo/apt)
make all       # build kernel + Debian rootfs, assemble the image
# result: image/de1soc-debian13-6.12.img
```

On any other Linux x86-64 host, `make docker-build` builds the same image inside a Docker container instead of `make deps + make all` (the host needs only Docker plus `qemu-arm` registered for ARM emulation; the target checks and explains). A build takes roughly 15–20 minutes. The Linux kernel source is a shallow git submodule pinned to v6.12; if you cloned without `--recurse-submodules`, `make` initialises it for you. See `hps/newimage/README.md` for details.

Keep a reference copy

Keep a known-good image untouched as a reference; always flash a copy.

3 Flash the microSD card

Write the whole `.img` to the raw card — not to a partition, and not as files.

Data-loss warning

Flashing overwrites the entire target device. Selecting the wrong disk erases it. Identify the card by its exact size and unplug other external drives first. With `dd`, double-check the device name every time.

3.1 Option A — balenaEtcher (any OS)

Flash from file → select the `.img` → *Select target* (the microSD card, confirm the size) → *Flash*. Etcher verifies automatically.

3.2 Option B — dd (Linux / macOS)

```
# Linux: find the card by size (e.g. /dev/sdX or /dev/mmcblkN)
lsblk
sudo umount /dev/sdX*      # unmount any auto-mounted partitions
sudo dd if=de1soc-debian13-6.12.img of=/dev/sdX bs=4M conv=fsync status=progress
sync
```

```
# macOS: note the raw rdisk node, and unmount first
diskutil list
diskutil unmountDisk /dev/diskN
sudo dd if=de1soc-debian13-6.12.img of=/dev/rdiskN bs=4m
sync
```

Sanity check

A genuine write of a ~1.8 GB image takes minutes, not seconds. If `dd` returns almost instantly, it did not reach the physical card — re-check the device name.

4 Set the boot-mode switch

The DE1-SoC has a small DIP switch bank (SW10) that sets the FPGA configuration mode via the MSEL[4:0] pins. The Linux image needs U-Boot to configure the FPGA, so all five MSEL bits must be **0**.

Required setting

Set SW10 to **MSEL[4:0] = 00000** (all five MSEL positions to the “0” side — check the board silkscreen / user manual for which direction is 0).

At boot, U-Boot loads the FPGA design (soc_system.rbf) over the passive (FPP) path. In any other MSEL mode the FPGA self-configures from on-board flash, the design will not match, and the kernel can hang early during device probe. HPS boot from SD is selected separately (BSEL) and is unaffected by MSEL.

5 Connect and power on

1. Insert the microSD card into the slot on the underside of the board (HPS side), pushed fully home.
2. Connect the console USB cable from the board’s UART-to-USB port to your computer.
3. Optional: connect an Ethernet cable for SSH/networking.
4. Plug in power — but open your serial terminal (next section) *before* switching on, so you catch the full boot log.

6 Open the serial console

The HPS console runs at **115200 baud, 8N1, no flow control**. First find which port the CP210x bridge enumerated as:

```
# Linux
ls /dev/ttyUSB*      # or: dmesg | grep -i cp210
# macOS
ls /dev/tty.usbserial* # or /dev/tty.SLAB_USBtoUART
# Windows: Device Manager -> Ports (COM & LPT) -> Silicon Labs CP210x
```

Two ports?

The bridge is dual-channel and enumerates *two* serial ports. The HPS console is on one of them (commonly the lower-numbered, e.g. ttyUSB0). If one shows nothing on boot, try the other.

```
minicom -b 115200 -D /dev/ttyUSB0
picocom -b 115200 /dev/ttyUSB0
screen /dev/ttyUSB0 115200
# Windows: PuTTY -> Serial, COMn, 115200 (or Tera Term)
```

Note

On Linux your user needs serial access: `sudo usermod -aG dialout $USER`, then log out and back in.

7 First boot and login

Switch the board on. With the card flashed and MSEL at 00000, it autoboots through: Boot ROM → vendor preloader (SDRAM init) → vendor U-Boot (runs `u-boot.scr`: loads the FPGA design, then the kernel) → kernel → Debian login. You should see:

```
=== DE1-SoC Debian 13 / Linux 6.12 ===
u-boot.scr: loading GHRD soc_system.rbf into the FPGA fabric
FPGA configured with soc_system.rbf
...
de1-soc-debian login:
```

Sign in with the image's default account:

```
user: debian
pass: tempwd      # the root account password is also tempwd
```

Do this first

Change the default passwords immediately with `passwd` — the defaults are public knowledge.

First run, to grow the root filesystem to fill the card:

```
sudo /usr/local/sbin/expand-rootfs.sh && sudo reboot
```

Built in and ready to use

The image ships a few conveniences out of the box: a **native C toolchain** (`gcc`, `make`) so small programs compile on the board (see *Cross-compile for the board*); **USB camera support** — plug a UVC webcam or a Sony PlayStation Eye (`gspca/ov534`) into the USB-OTG port and it appears as `/dev/video0` for V4L2 capture; and **runtime FPGA reconfiguration** from Linux (see *Changing the FPGA fabric*).

8 Networking and SSH

The Ethernet interface is `end0` (the kernel's name for it; not `eth0`). It is configured for DHCP and an SSH server starts at boot. Once Ethernet has an address you can drop the serial cable.

```
# on the board
ip addr show end0      # look for an inet address
# from your computer
ssh debian@de1-soc-debian # by hostname, or use the IP address
```

No DHCP / direct link?

Connecting the board straight to a computer: assign static IPs on both ends of the link (same subnet), set the board's default gateway to the computer, and enable IP forwarding + NAT on the computer's uplink to share its internet.

9 The U-Boot prompt

You rarely need it, but U-Boot is where you change boot behaviour — kernel arguments, loading a different FPGA design, alternate boot sources. Press any key during the “Hit any key to stop autoboot” countdown; inspect and change with `printenv`, `setenv`, `saveenv`; resume with `boot`. The rootfs is the third partition (`/dev/mmcblk0p3`); the FAT boot partition is the first. `u-boot.scr`

(sourced from the FAT partition) sets `mmccroot` and loads `soc_system.rbf`, so you normally change boot behaviour by editing that script rather than the saved environment.

```
# inspect the environment / the root device the script selects
printenv mmccroot          # -> /dev/mmcb1k0p3
```

10 Cross-compile for the board

The HPS is a 32-bit ARM Cortex-A9 running hard-float Debian (armhf).

```
# Debian/Ubuntu host: install the toolchain
sudo apt install gcc-arm-linux-gnueabi g++-arm-linux-gnueabi
arm-linux-gnueabi-gcc --version
```

```
# compile, then copy to the board and run
arm-linux-gnueabi-gcc -O2 -o hello hello.c
scp hello debian@de1-soc-debian:~
ssh debian@de1-soc-debian ./hello
```

Match the target's libraries

The image is Debian 13 (glibc 2.41). Build with a toolchain of the same era or older; a much newer cross-toolchain can link against glibc symbols the board lacks (version 'GLIBC_2.xx' not found at runtime). For small tools link statically (`-static`); for larger apps build against a sysroot copied from the board (`--sysroot`). The image ships `gcc` and `make` preinstalled, so you can also compile small programs natively on the board (add `apt install build-essential` for the full toolset).

To build a kernel module, use the kernel source tree this repo builds (matching 6.12.0):

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- \
-C hps/newimage/kernel/linux-6.12 M=$(pwd) modules
```

11 Build your own FPGA design

The fabric is a Cyclone V SoC (5CSEMA5F31C6). Design it in Intel Quartus Prime with Platform Designer, export a raw bitstream (`.rbf`), and place it on the card so U-Boot configures the FPGA at boot. The reference design and the recipe that produced the shipped `soc_system.rbf` are in `fpga/hps_ghrd/` (see its README).

11.1 Getting the tools

- **Quartus Prime Lite Edition** — the free edition (no license required) that supports Cyclone V. Download it from the vendor's FPGA software download center: historically the *Intel FPGA Software Download Center* (fpgasoftware.intel.com); since Altera was spun back out of Intel (2024–2025) it is served from [altera.com](https://www.altera.com) — search “Quartus Prime Lite download”. During installation, include **Cyclone V device support**.
- **Reference design (GHRD)** — in this repo at `fpga/hps_ghrd/`, and on the DE1-SoC System CD; the recommended starting point.

You do not need Quartus to run the OS image

The compiled bitstream is committed (`hps/newimage/fpga/soc_system.rbf`). Quartus is only needed to *rebuild or modify* the fabric.

Start from the reference design

Do not begin from a blank project. The Golden Hardware Reference Design (GHRD) — a working DE1-SoC project with the correct device, top-level, pin assignments, and HPS/DDR3 setup — is in `fpga/hps_ghrd/`. Modify it rather than recreate it. See `fpga/hps_ghrd/README.md` for the full `qsys-generate` → `compile` → `quartus_cpf` recipe.

```
# regenerate the Qsys system, compile, then convert .sof -> raw UNCOMPRESSED .rbf
cd fpga/hps_ghrd
qsys-generate de1_soc.qsys --synthesis=VERILOG
quartus_sh --flow compile de1_soc_top_v2
quartus_cpf -c -o bitstream_compression=off \
  output_files_v2/de1_soc_top_v2.sof soc_system.rbf
cp soc_system.rbf ../../hps/newimage/fpga/soc_system.rbf
```

The bitstream lives on the FAT boot partition as `soc_system.rbf`, loaded by `u-boot.scr`. Card-swap: power off → card to reader → replace `soc_system.rbf` → card back → power on. (For MSEL=00000/FPP the `.rbf` must be **uncompressed**, ~7 MB — see the warning below.)

Keep the bitstream and device tree in sync

If you change which peripherals exist or their addresses, update the kernel device tree (`socfpga.dtb` on the FAT partition) to match. A kernel whose device tree references fabric hardware that is not in the loaded bitstream can hang during driver probe.

12 Changing the FPGA fabric

There are two ways to change the fabric on this image:

- **At boot** — U-Boot's `u-boot.scr` loads `soc_system.rbf` from the FAT partition over the FPP path. Replace that file on the FAT partition (above) and reboot; the change is permanent.
- **At runtime from Linux** — `fpga-load.sh` reprograms the fabric without a reboot (see below). The change lasts until the next reboot, when U-Boot loads `soc_system.rbf` again.

The .rbf must be uncompressed

With MSEL=00000 the HPS configures the FPGA over the passive (FPP) path, which has compression disabled. A compressed `.rbf` (`-o bitstream_compression=on`) streams in but the FPGA cannot expand it, so configuration never completes (`err=-110`). Generate it uncompressed: `quartus_cpf -c -o bitstream_compression=off design.sof design.rbf`. Sanity check: an uncompressed Cyclone V (5CSEMA5) image is ~7 MB; a ~2 MB file is compressed.

Runtime reconfiguration from Linux (no reboot)

You can also reconfigure the fabric at runtime from the HPS. Mainline Linux has no built-in userspace overlay interface, so this image autoloads the `dtbocfg` module (providing `/sys/kernel/config/device-tree/overlays`) and ships `/usr/local/sbin/fpga-load.sh`. It applies a device-tree overlay to `/soc/base_fpga_region` naming a bitstream under `/lib/firmware` (the overlay is compiled on the board with the preinstalled `dtc`), so the FPGA Manager programs the fabric. Two demos ship ready to load in `/lib/firmware/designs/`:

```
sudo fpga-load.sh --name designs/marquee.rbf # scrolls "debian 13" on HEX5..HEX0
sudo fpga-load.sh --name designs/countdown.rbf # counts 100 -> 0 on HEX2..HEX0
```

```
cat /sys/class/fpga_manager/fpga0/state      # -> operating
sudo fpga-load.sh -u                        # remove the overlay
```

For your own design, pass the .rbf: `sudo fpga-load.sh my_design.rbf` copies it to `/lib/firmware/fpga.rbf` and loads it; `--link my_design.rbf` symlinks it there instead of copying; and `--name <file>` loads any file under `/lib/firmware` by name. `sudo fpga-load.sh -s` shows what is loaded. The .rbf must be uncompressed either way. See `fpga/fpga-load/`.

This works for any design, including ones that expose memory-mapped slaves to the HPS through the FPGA bridges: the FPGA region disables the bridges before reprogramming and re-enables them afterwards, so the fabric side re-synchronises with no HPS hang.

This image is headless (no VGA frame buffer — the mainline kernel has no `altvipfb` driver), so there is no running display to disturb when you change the fabric, as there was on the old desktop image.

13 Troubleshooting

Symptom	Likely cause & fix
Nothing on the serial console	Wrong port (try the other CP210x channel, e.g. <code>ttyUSB1</code>), wrong baud (must be 115200), not in the <code>dialout</code> group, driver not installed, or cable in the wrong USB jack.
Garbled characters	Baud mismatch — set exactly 115200 8N1, no flow control.
U-Boot runs, freezes just after Starting kernel...	MSEL is not 00000, so the FPGA lacks the design the kernel expects. Set <code>MSEL[4:0]=00000</code> and power-cycle.
No boot / boot loops / “card did not respond”	Image not written correctly. Re-flash and verify; confirm the card is fully seated.
<code>hostnamectl</code> : “Failed to connect to bus”	D-Bus not running — on an older image: <code>sudo apt install -y dbus</code> ; the current image includes it.
<code>journalctl</code> : “insufficient permissions”	<code>sudo usermod -aG systemd-journal debian</code> and re-login (the current image already does this).
Clock/time is wrong	The board has no RTC; it sets over NTP once networked (<code>systemd-timesyncd</code>).
<code>err=-110</code> on FPGA Manager apply	The .rbf is compressed. Regenerate uncompressed: <code>quartus_cpf -c -o bitstream_compression=off design.sof design.rbf</code> .
No IP on <code>end0</code>	Ensure the network has DHCP; check <code>ip addr</code> , or set a static address.